Task 1

1. the file have rwxr-xr-x permeation so i want to change it to owner read /write, the group read-only and the others have no access

- so the 3-digit command is

```
chmod 640 <file name>
```

2. the executioner we have a script with permeation 644 now we want to make it excitable for every one while keeping the other permeation

```
chmod 755 <file name>
```

3. log file needs to be writable by the owner and group but readable by every one

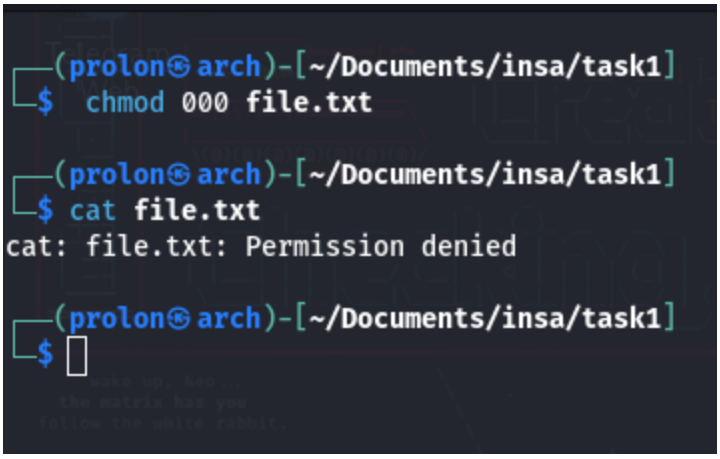
```
chmod 664 <file name>
```

4. binary to octal the file binary permeation string is 110 100 000 so the chmod numeric command is

```
chmod 640 <file name>
```

5. the mystery code what will happens to a file if we run chmod 000 to file can the owner still cat the without sudo

- no cuz he doesn't have the privilege to prof it



6. create a file named "Audit.txt" and lets assign by the following permeation `rwxr-x---`

```
┌──(prolon㉿arch)-[~/Documents/insa/task1]
└─$ touch Audit.txt

┌──(prolon㉿arch)-[~/Documents/insa/task1]
└─$ ls -l
total 4
-rw-rw-r-- 1 prolon prolon 0 Mar 21 17:24 Audit.txt
---------- 1 prolon prolon 5 Mar 21 16:47 file.txt

┌──(prolon㉿arch)-[~/Documents/insa/task1]
└─$ chmod u=rwx,g=rx,o= Audit.txt

┌──(prolon㉿arch)-[~/Documents/insa/task1]
└─$ ls -l
total 4
-rwxr-x--- 1 prolon prolon 0 Mar 21 17:24 Audit.txt
---------- 1 prolon prolon 5 Mar 21 16:47 file.txt

┌──(prolon㉿arch)-[~/Documents/insa/task1]
└─$
```

7. lets change numeric `741` to symbolic

- `rwxrw---x`

8. what is the equivalent of 500 permeation in symbolic

- `r-x------`

9. what is the numeric equivalent of -r-xrw-r--

- `562`

Task 2

challenge.jpg

- There are 4 flags hidden inside the image provided below in the format FLAG{...}
- Hint:

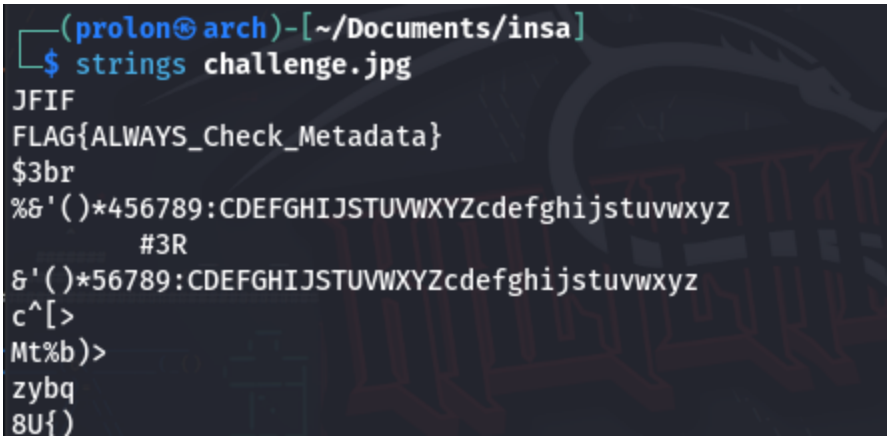
1. The first 2 flags require implementation of today's class on extracting information from a data.
2. To retrieve the remaining 2, read on Stenography and what tools to use to extract information from an image
3. 5th flag!

- to get the first two flags i use the strings command **the strings command in kali is used to extract readable text (ASCII strings) from the binary files it scan files like executable, memory damps, or malware samples and prints human-readable text hidden inside them

```
strings <file name>
```

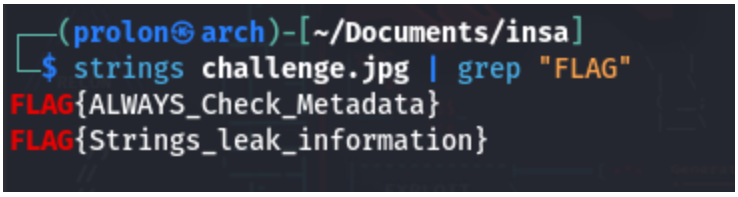
so in this way i use the strings command to see the out put in the image file

```
strings challenge.jpg
```



so as we see the out put of this command is now we want the flags so lets grep

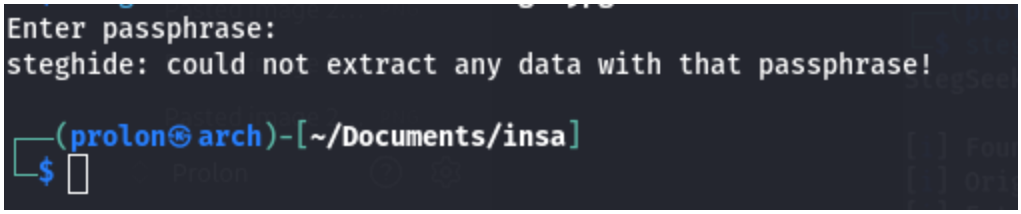
```
strings challenge.jpg | grep "FLAG"
```



booooom! we got the 2 flags

```
FLAG{ALWAYS_Check_Metadata}
FLAG{Strings_leak_information}
```

now lets find the other two for the other two it use's stenography so let's try the tools first tool stateside



we don't the passphrase so let's try other tools or let's brute force

- we use the stegseek tool to brute force

```
┌──(prolon㉿arch)-[~/Documents/insa]
└─$ stegseek challenge.jpg  /usr/shere/wordlists/rockyou.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[!] error: could not open the wordlist "/usr/shere/wordlists/rockyou.txt".

┌──(prolon㉿arch)-[~/Documents/insa]
└─$ stegseek challenge.jpg  /usr/share/wordlists/rockyou.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "password123"
[i] Original filename: "flag5.txt".
[i] Extracting to "challenge.jpg.out".


┌──(prolon㉿arch)-[~/Documents/insa]
└─$ ls
 1                              finalflag.bin
 challenge.jpg                  hidden.bin
 _challenge.jpg-0.extracted     hidden.zip
 _challenge.jpg-1.extracted    'Linux Fundamentals - cheatsheet.pdf'
 _challenge.jpg-2.extracted     stegsolve.jar
 _challenge.jpg-3.extracted     task
 _challenge.jpg-4.extracted     task1
 _challenge.jpg.extracted       yay
 challenge.jpg.out

┌──(prolon㉿arch)-[~/Documents/insa]
└─$ cat challenge.jpg.out
FLAG{steghide_protected}
```

- ok now we have the first second and the fifth flag so lets chacke the others 2
- for now lets try binwalk tool

```
┌──(prolon㉿arch)-[~/Documents/insa]
└─$ binwalk -Me challenge.jpg

Scan Time:     2026-03-22 02:23:38
Target File:   /home/prolon/Documents/insa/challenge.jpg
MD5 Checksum:  2da1e6db0a36e3de001d42e13f69aacc
Signatures:    436

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
75667         0x12793         Zip archive data, at least v1.0 to extract, compressed size: 41, uncompressed size: 41, name: flag4.txt

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented


Scan Time:     2026-03-22 02:23:38
Target File:   /home/prolon/Documents/insa/_challenge.jpg-5.extracted/flag4.txt
MD5 Checksum:  ff58b44f7e20695ab02140ba5bc2472b
Signatures:    436

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------


┌──(prolon㉿arch)-[~/Documents/insa]
└─$ ls
 1                          _challenge.jpg-1.extracted   _challenge.jpg-4.extracted   challenge.jpg.out   hidden.bin                          stegsolve.jar   yay
 challenge.jpg              _challenge.jpg-2.extracted   _challenge.jpg-5.extracted   finalflag.bin       hidden.zip                          task
 _challenge.jpg-0.extracted _challenge.jpg-3.extracted   _challenge.jpg.extracted     flag5.txt          'Linux Fundamentals - cheatsheet.pdf'  task1

┌──(prolon㉿arch)-[~/Documents/insa]
└─$ cd _challenge.jpg-5.extracted

┌──(prolon㉿arch)-[~/Documents/insa/_challenge.jpg-5.extracted]
└─$ ls
```

```
┌──(prolon㉿arch)-[~/Documents/insa/_challenge.jpg-5.extracted]
└─$ ls
12793.zip  flag4.txt

┌──(prolon㉿arch)-[~/Documents/insa/_challenge.jpg-5.extracted]
└─$ cat falg
cat: falg: No such file or directory

┌──(prolon㉿arch)-[~/Documents/insa/_challenge.jpg-5.extracted]
└─$ cat falg4.txt
cat: falg4.txt: No such file or directory

┌──(prolon㉿arch)-[~/Documents/insa/_challenge.jpg-5.extracted]
└─$ cat flag4.txt
Umt4QlIzdGhjSEJsYm1SbFpGOTZhWEI5Q2c9PQo=

┌──(prolon㉿arch)-[~/Documents/insa/_challenge.jpg-5.extracted]
└─$
```

now we have the base64 code so lets decode it

```
┌──(prolon㉿arch)-[~/Documents/insa/_challenge.jpg-5.extracted]
└─$ cat flag4.txt | base64 -d | base64 -d
FLAG{appended_zip}

┌──(prolon㉿arch)-[~/Documents/insa/_challenge.jpg-5.extracted]
└─$
```

for the last flag i use a simple python script to extract it

```
data = open('challenge.jpg', 'rb').read()
for key in range(256):
    decoded = ''.join(chr(b ^ key) if 32 <= (b ^ key) <= 126 else '.' for b
in data)
    if 'FLAG' in decoded:
        start = decoded.find('FLAG')
        print(f'Key {hex(key)} found Match: {decoded[start:start+40]}')
```

so let's see if it works

```
┌──(prolon㉿arch)-[~/Documents/insa]
└─$ chmod +x chalenge.py

┌──(prolon㉿arch)-[~/Documents/insa]
└─$ python3 chalenge.py
Key 0x0 found Match: FLAG{ALWAYS_Check_Metadata}...C.........
Key 0x20 found Match: FLAG..TXTut) #^..I^..IUX+ !$.#  $.#  uMT
Key 0x42 found Match: FLAG{xor_hidden}..AFHBBBBB..7.....kBBBkB

┌──(prolon㉿arch)-[~/Documents/insa]
└─$
```

so we have found the last flag

```
FLAG{ALWAYS_Check_Metadata}
FLAG{Strings_leak_information}
FLAG{appended_zip}
FLAG{steghide_protected}
FLAG{xor_hidden}
```